Lecture 1 – Course logistics, all about data!

Ahmad El-Hajj, PhD

Course logistics

My info

- Name : Dr. Ahmad El Hajj
- Email: ae37@aub.edu.lb
- Office hours: MW 10:00-11:00 or anytime by appointment
- Office: OSB 367

Course description (dissected)

The course focuses on the technology processing of data prior to performing data analytics to extract actionable intelligent insights. It covers data preparation, integration and processing using open-source software tools such as Python, Hadoop and Spark based platforms. Topics include identification of the datasets of interest; preparing the data; building data models using SQL and NoSQL databases and performing operations to explore large and complex datasets. It also explores big data and how and when to integrate big data platforms within an enterprise information system. Through guided hands-on tutorials, students will become familiar with data processing techniques using different frameworks.

Topical coverage

Week	Topic*	Learning Activities** and Due Dates
1	General: Data, Types, and Databases	Reading: Overview of data types and databases.
2	Data Lifecycle and Curation	Hands-on exercise: Dataset identification and data curation (due Week 2).
3	Data Cleaning Techniques	Assignment 1: Preprocessing real-world datasets (due Week 4).
4	Exploratory Data Analysis	Lab: EDA with Python and visualization tools.
5	Further Data analysis	Data validation techniques, data quality metrics, and governance basics.
6	Relational Databases (SQL)	Quiz 1: SQL basics and queries (Week 6).
7	NoSQL Databases and MongoDB	Lab: Designing a NoSQL database in MongoDB.
8	Graph Databases and Neo4j	Assignment 2: Building a knowledge graph in Neo4j (due Week 8).
9	Big Data Platforms Overview	Reading: Introduction to Databricks and Spark.
10	Data Processing with Databricks	Lab: Scalable data transformation on Databricks.
11	Knowledge Graph Construction	Hands-on project: Constructing and querying knowledge graphs.
12	Integrating Data Security	Reading: Data security best practices.
13	Cloud Services for Data Management	Lab: Deploying data pipelines in the cloud.
14	Applications of Knowledge Graphs	Case study: Using knowledge graphs in business analytics.
15	Course Review and Final Quiz	Quiz 2: Comprehensive quiz (Week 15).

Grading

Type of Evaluation	Percentage
Assignments	20%
Quiz 1	15%
Project	35%
Final Exam	20%
Attendance/Participation	10%

More about grades

- You don’t order grades like you order a pizza
- I don’t have a magic wand !!

- Once the grade is out, I can’t reply to such emails:
  - I need to pass
  - I need more grades to keep the financial aid
  - Dr. , I can do whatever you want to pass

- Your grade on the system is the maximum I could have given to you !!

AI Policy

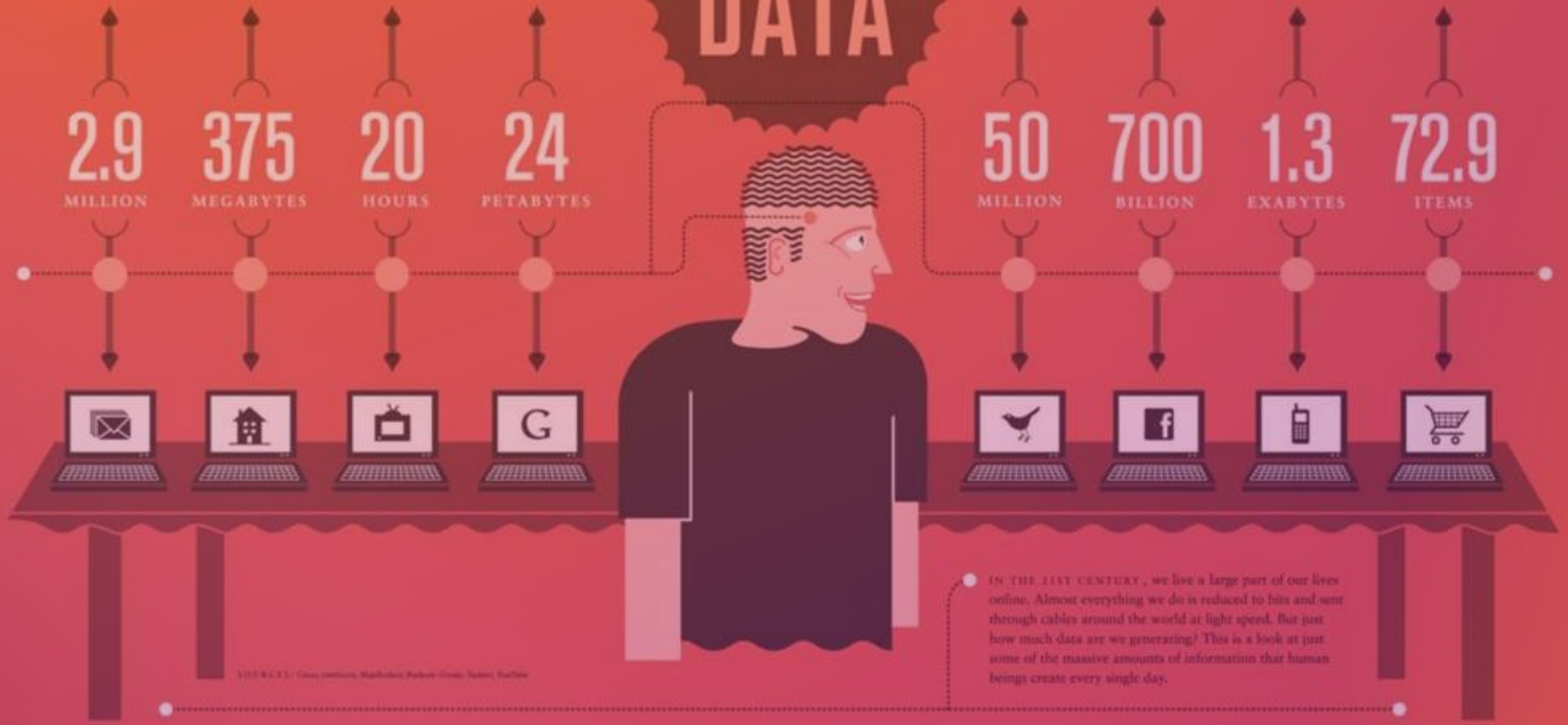
The use of generative AI tools is only allowed for specific parts of this course which include assignments, and additional learning. For assignments that require the use of AI tools, the students will be guided on the requirements and how these tools should be employed. Students can also use generative AI tools such as ChatGPT, Gemini, or Claude to improve the attainment of the course learning outcomes.

On another note, generative AI tools are strictly prohibited in exams, quizzes and any other component where the course instructor clearly indicates that generative AI tools should not be employed. Any use of generative AI tools beyond these outlined permissions, rules and expectations, will be deemed as academic dishonesty and a breach of the AUB Student Code of Conduct.

Data, Types, and Databases

THE WORLD OF DATA

NUMBER OF EMAILS SENT EVERY SECOND	DATA CONSUMED BY HOUSEHOLDS EACH DAY	VIDEO UPLOADED TO YOUTUBE EVERY MINUTE	DATA PER DAY PROCESSED BY GOOGLE	TWEETS PER DAY	TOTAL MINUTES SPENT ON FACEBOOK EACH MONTH	DATA SENT AND RECEIVED BY MOBILE INTERNET USERS	PRODUCTS ORDERED ON AMAZON PER SECOND
2.9 MILLION	375 MEGABYTES	20 HOURS	24 PETABYTES	50 MILLION	700 BILLION	1.3 EXABYTES	72.9 ITEMS



IN THE 21ST CENTURY, we live a large part of our lives online. Almost everything we do is reduced to bits and sent through cables around the world at light speed. But just how much data are we generating? This is a look at just some of the massive amounts of information that human beings create every single day.

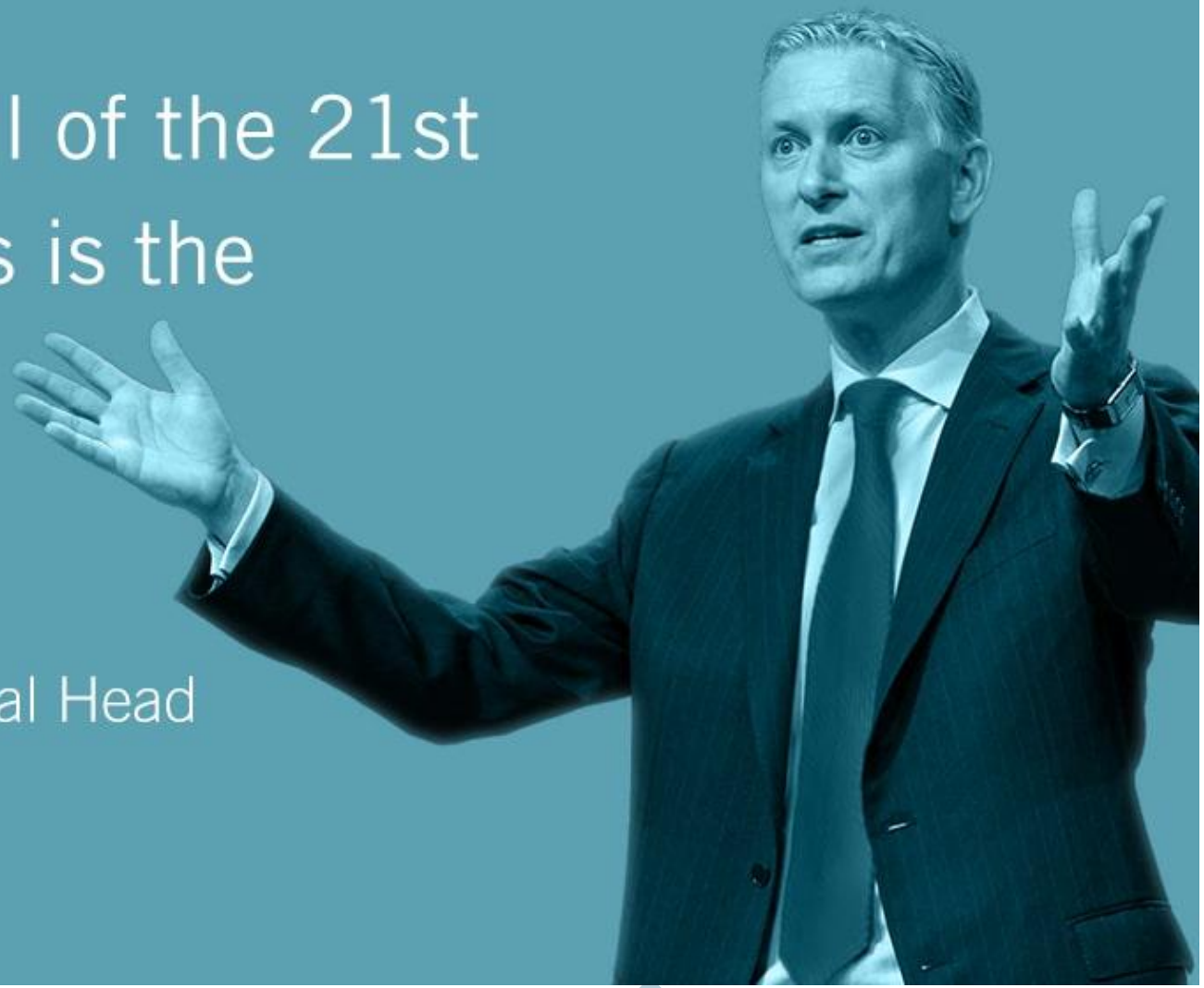
VISUALS: Cisco (estimate), MapReduce, Radicati Group, Twitter, YouTube

A COLLABORATION BETWEEN GOOD AND OLIVER MUNDAY

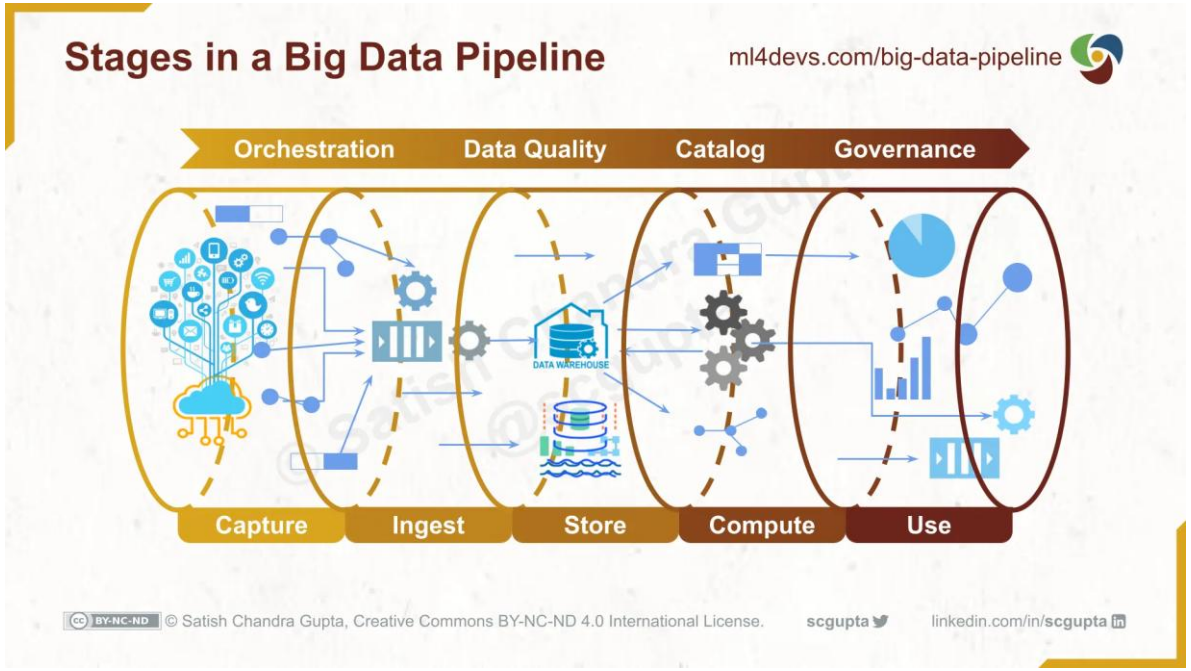
IN PARTNERSHIP WITH IBM

"Information is the oil of the 21st century, and analytics is the combustion engine."

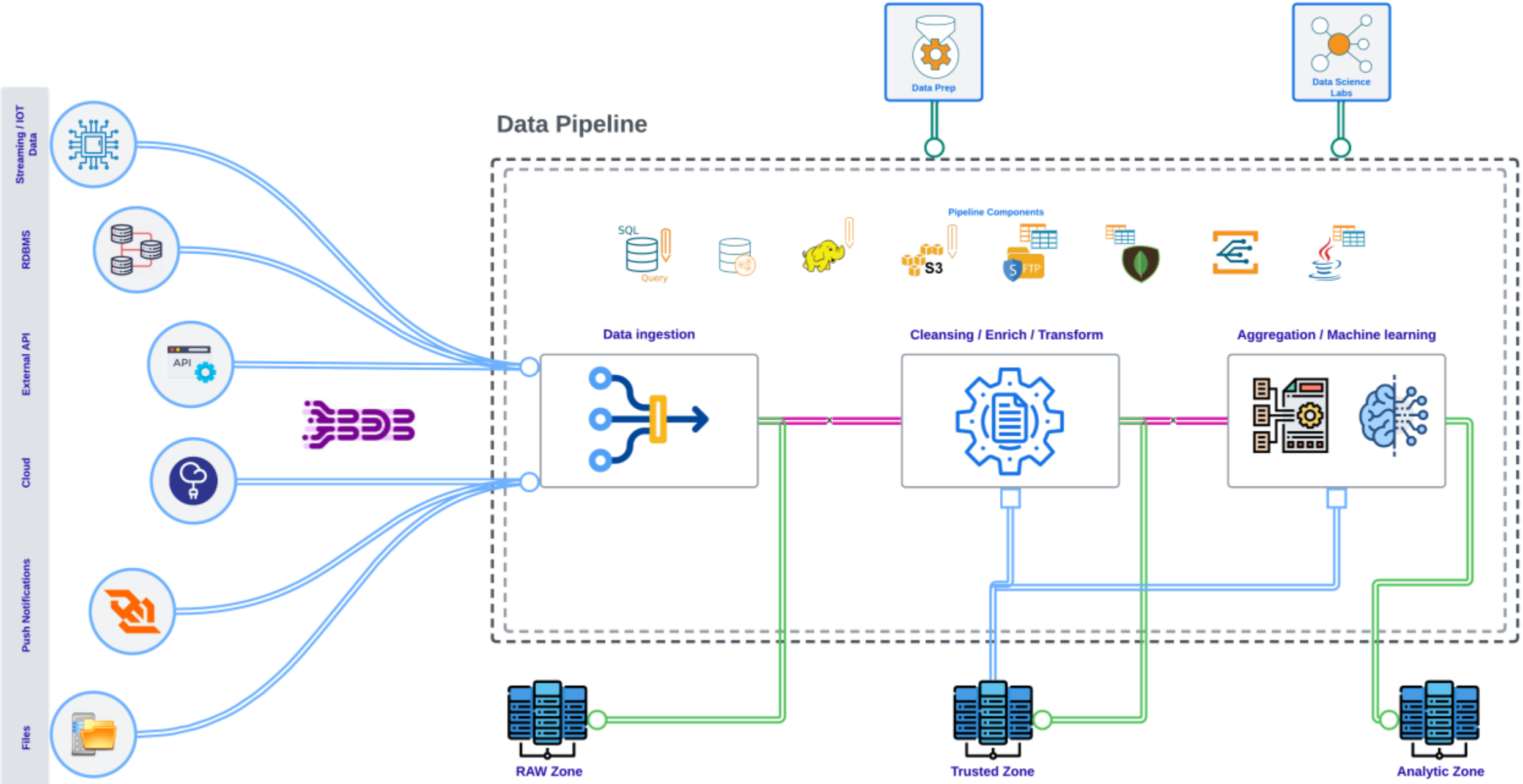
Peter Sondergaard
Senior Vice President and Global Head of Research at Gartner, Inc.



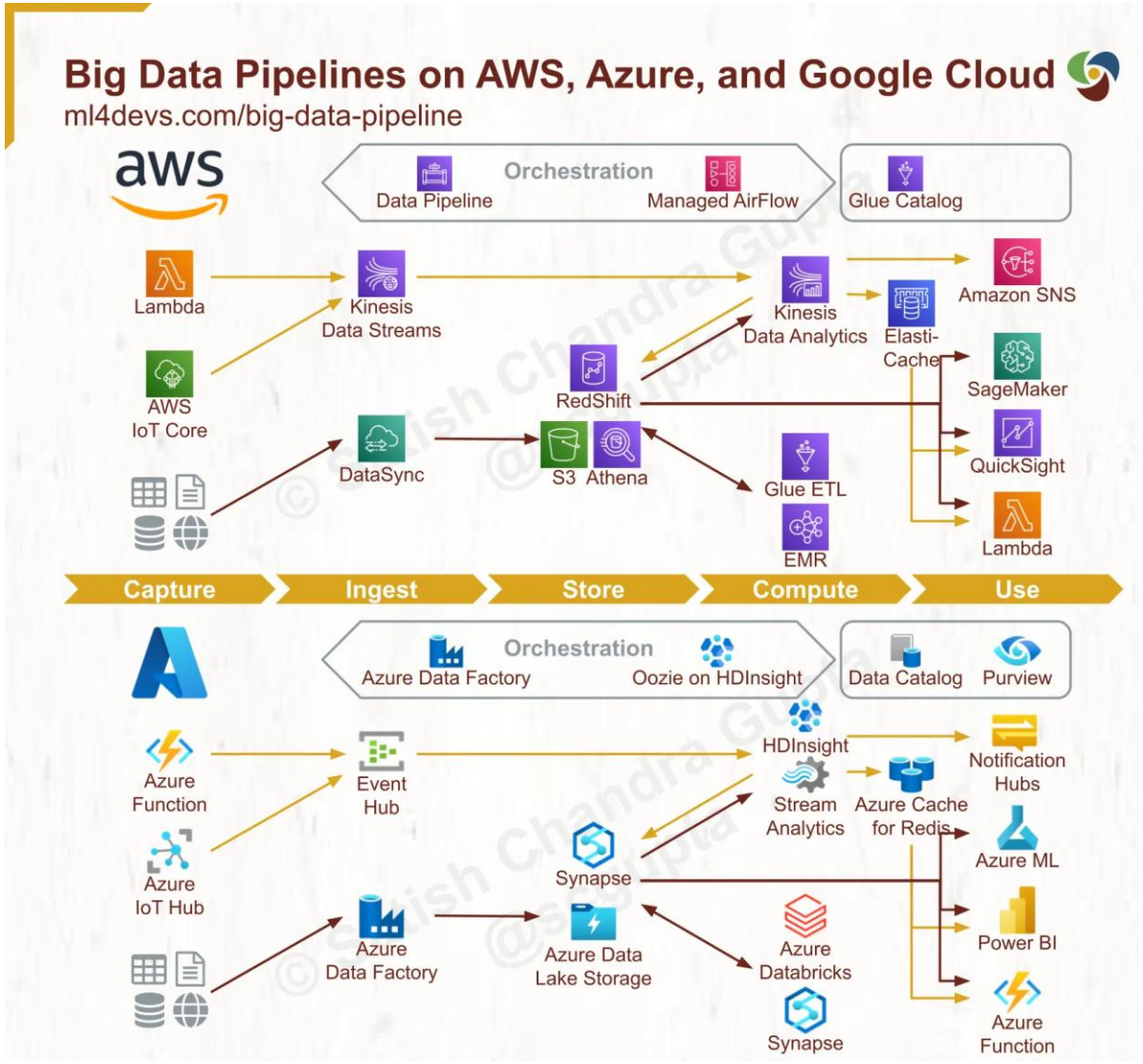
Stages of data processing

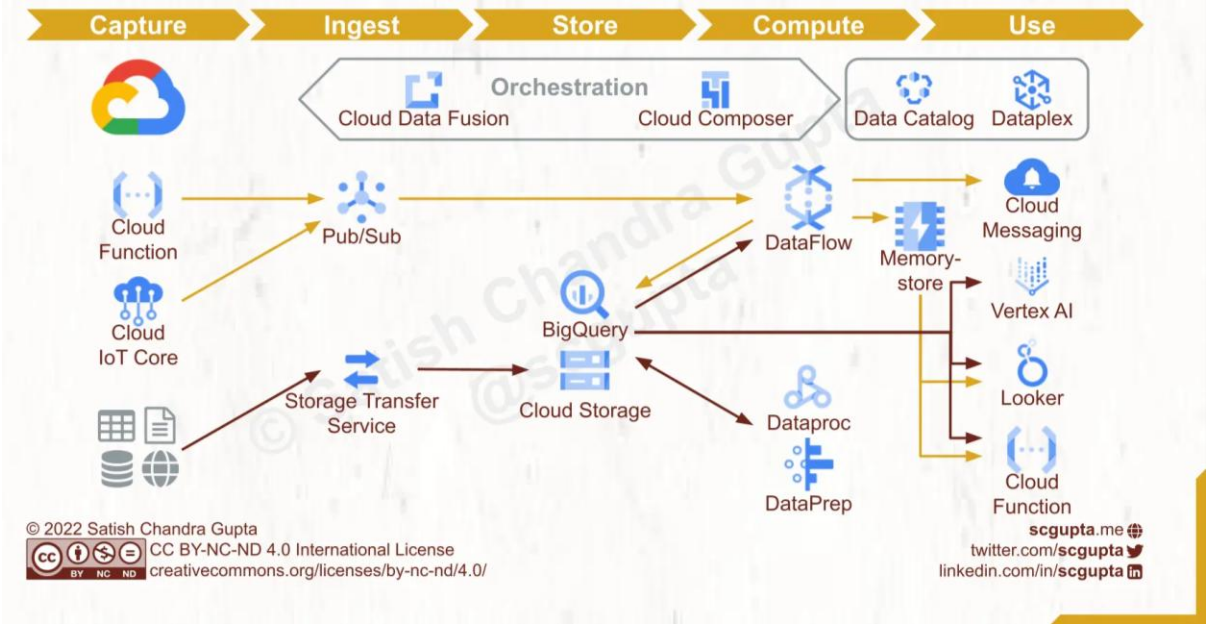


Capture Data from Internal & External Sources
•Instrumentation of data sources: mobile apps, websites, web apps, microservices, IoT devices, etc.
•Captures relevant data for analysis and processing.
Ingest Data Through Batch Jobs or Streams
•Data flows via HTTP, MQTT, or message queues.
•Jobs import data from sources like Google Analytics.
•Data forms: batch blobs and streams.
Store in Data Lake or Data Warehouse
•Raw data/events stored in Data Lakes.
•Cleaned, deduplicated, and transformed data moved to Data Warehouses.
Compute Analytics Aggregations and/or ML Features
•Analytics, data science, and machine learning processes occur here.
•Batch and stream processing generate models and insights.
•Results are stored in the Data Warehouse.
Use in Dashboards, Data Science, and ML
•Deliver insights via dashboards, emails, SMSs, push notifications, and microservices.
•ML model inferences exposed as microservices.
Orchestration
•Data pipelines form Directed Acyclic Graphs (DAGs).
•Orchestration ensures correct sequencing and dependency management.
Data Quality
•Checks statistical distribution, outliers, and anomalies at each pipeline stage.
Catalog
•Data Catalog provides metadata and schema for better understanding of data assets.
Data Governance
•Ensures security, anonymization, accuracy, and availability throughout the data lifecycle.

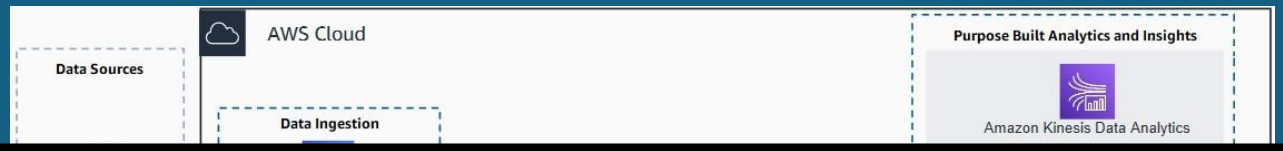

Data Prep
Data Science Labs
Data Pipeline
Streaming / IOT Data
RDBMS
External API
Cloud
Push Notifications
Files
API
Pipeline Components
SQL
Query
S3
FTP
Data ingestion
Cleansing / Enrich / Transform
Aggregation / Machine learning
RAW Zone
Trusted Zone
Analytic Zone

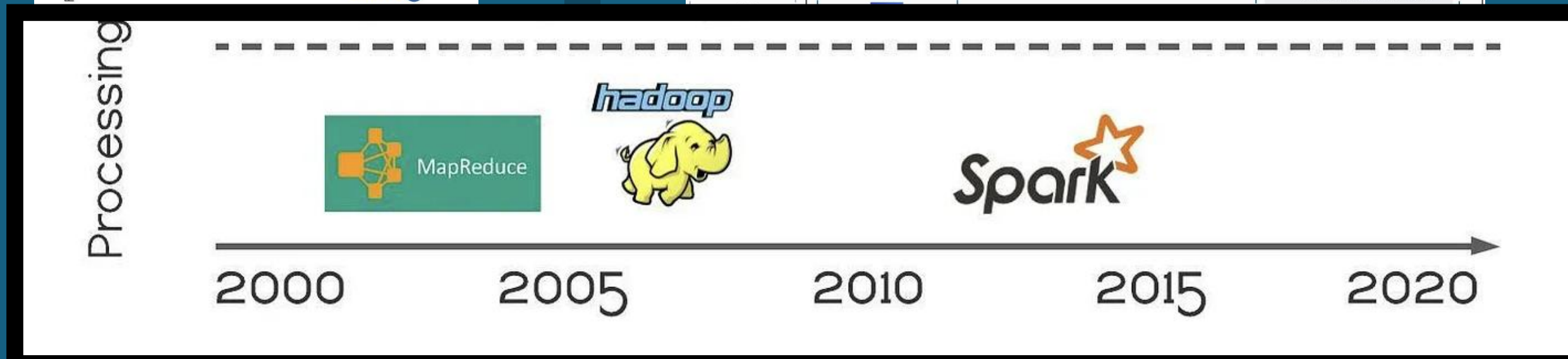
Embracing the cloud





Data Formats
Table Formats
AVRO
Parquet
ARROW
Apache hudi
DELTA LAKE
ICEBERG

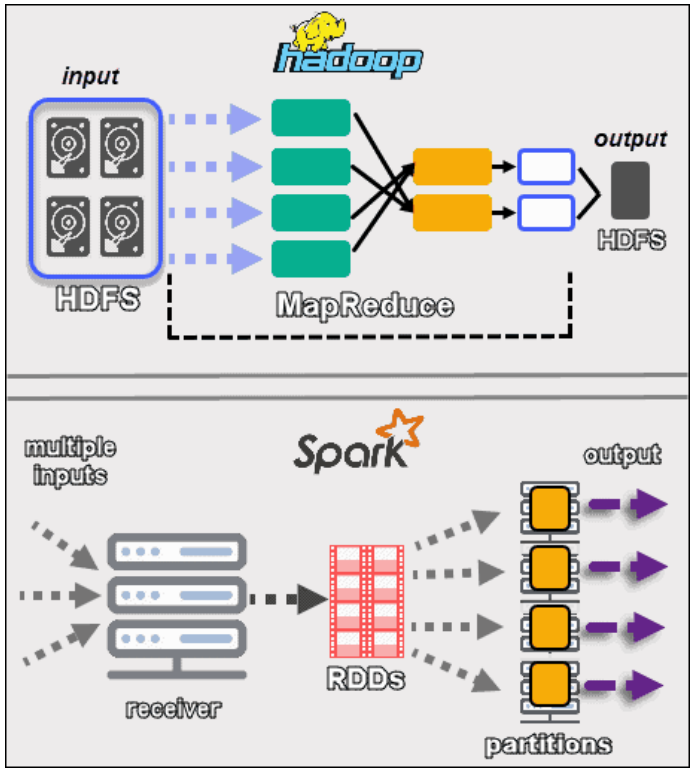

AWS Cloud
Data Sources
Data Ingestion
Purpose Built Analytics and Insights
Amazon Kinesis Data Analytics


Processing
MapReduce
hadoop
Spark
2000
2005
2010
2015
2020

AWS Data Exchange
Amazon Redshift Spectrum Amazon Athena

Coming up in the course…

Feature	Hadoop MapReduce	Apache Spark
Processing Model	• Processes data in sequential batches • Jobs divided into map and reduce tasks • Writes intermediate results to disk	• Flexible processing including batch, streaming, and iterative • Uses in-memory computation • Supports complex multi-stage pipelines
Data Storage	• Uses Hadoop Distributed File System • Data split into blocks and replicated • Strong data durability focus	• Supports multiple data sources • Works with distributed and local storage • Compatible with Hadoop, Amazon S3, Cassandra
Fault Tolerance	• Achieves reliability through data replication • Restarts failed tasks automatically • Retrieves data from replicated copies	• Uses Resilient Distributed Datasets tracking • Rebuilds lost data through transformation history • Reduces storage overhead through lineage
Performance Speed	• Moderate speed due to disk operations • Best for handling large batch jobs • Limited by disk read/write speed	• High speed with in-memory processing • Excellent for iterative algorithms • Significantly faster for repeated operations
Development Interface	• Primary Java programming interface • Supports high-level tools like Hive and Pig • Complex code required for basic tasks	• Multiple programming language support • Includes Java, Python, Scala, and R • Simple, intuitive programming interface
Real-time Processing	• No built-in streaming support • Requires additional frameworks • Designed for batch processing only	• Native streaming processing support • Handles live data processing • Processes data in micro-batches
Resource Management	• Fixed resource allocation • Limited flexibility in resource sharing • Requires manual configuration	• Flexible cluster management options • Dynamic resource allocation • Better resource utilization
Primary Use Cases	• Large data batch processing • Extract, Transform, Load operations • Historical data analysis • Log file processing	• Interactive data analysis • Machine learning applications • Real-time analytics • Graph data processing
Memory Usage	• Minimal memory requirements • Disk-based storage focus • Predictable memory patterns	• Higher memory requirements • Caches data in memory • Memory-optimized operations



Data sources

Data formats

TYPE OF DATA	PREFERRED FILE FORMATS FOR SHARING, RE-USE AND PRESERVATION	Other Acceptable formats
Quantitative tabular data with extensive metadata •a dataset with variable labels, code labels, and defined missing values, in addition to the matrix of data	•SPSS portable format (.por) •delimited text and command ('setup') file •(SPSS, Stata, SAS, etc.) containing metadata information •structured text or mark-up file containing metadata information, e.g. DDI XML file	MS Access (.mdb/.accdb)
Quantitative tabular data with minimal metadata •a matrix of data with or without column headings or variable names, but no other metadata or labelling	•comma-separated values (CSV) file (.csv) •tab-delimited file (.tab) •including delimited text of given character set with SQL data definition statements where appropriate	•delimited text of given character set -- only characters not present in the data should be used as delimiters (.txt) •widely-used formats, e.g. MS Excel (.xls/.xlsx), MS Access (.mdb/.accdb), dBase (.dbf) and OpenDocument Spreadsheet (.ods)
Geospatial data vector and raster data	•ESRI Shapefile •(essential: .shp, .shx, .dbf ; optional: .prj, .sbx, .sbn) •geo-referenced TIFF (.tif, .tfw) •CAD data (.dwg) •tabular GIS attribute data	•ESRI Geodatabase format (.mdb) •MapInfo Interchange Format (.mif) for vector data
Qualitative data textual	•eXtensible Mark-up Language (XML) text according to an appropriate Document Type Definition (DTD) or schema (.xml) •Rich Text Format (.rtf) •plain text data, UTF-8 (Unicode; .txt)	•plain text data, ASCII (.txt) •Hypertext Mark-up Language (HTML) (.html) •widely-used proprietary formats, e.g. MS Word (.doc/.docx) •LaTeX (.tex)
Digital image data	TIFF version 6 uncompressed (.tif)	•JPEG (.jpeg, .jpg) •TIFF (other versions; .tif, .tiff) •JPEG 2000 (.jp2) •Adobe Portable Document Format (PDF/A, PDF) (.pdf)
Digital audio data	•Free Lossless Audio Codec (FLAC) (.flac) •Waveform Audio Format (WAV) (.wav) •MPEG-1 Audio Layer 3 (.mp3) - spoken word audio only	•MPEG-1 Audio Layer 3 (.mp3) •Audio Interchange File Format (AIFF) (.aif)
Digital video data	•MPEG-4 High Profile (.mp4) •motion JPEG 2000 (.jp2)	JPEG 2000 (.mj2)
Documentation & Scripts	•Rich Text Format (.rtf) •Open Document Text (.odt) •HTML (.htm, .html)	•plain text (.txt) •widely-used proprietary formats, e.g. MS Word (.doc/.docx) or MS Excel (.xls/ .xlsx) •XML marked-up text (.xml) according to an appropriate DTD or schema, e.g. XHMTL 1.0 •PDF/A or PDF (.pdf)
Chemistry data spectroscopy data and other plots which require the capability of representing contours as well as peak position and intensity	Convert NMR, IR, Raman, UV and Mass Spectrometry files to JCAMP format for ease in sharing. JCAMP file viewers: JSpecView, ChemDoodle	

Oregon State University

THE SIX Vs OF BIG DATA

VOLUME

The amount of data from myriad sources

VARIETY

The types of data: structured, semi-structured, unstructured

VELOCITY

The speed at which big data is generated

VERACITY

The degree to which big data can be trusted

VALUE

The business value of the data collected

VARIABILITY

The ways in which big data can be used and formatted

Structured Data

- Highly organized information
- Can be seamlessly integrated into a database
- Can be searched very easily

Attribute, feature, variable, property

Simple attribute

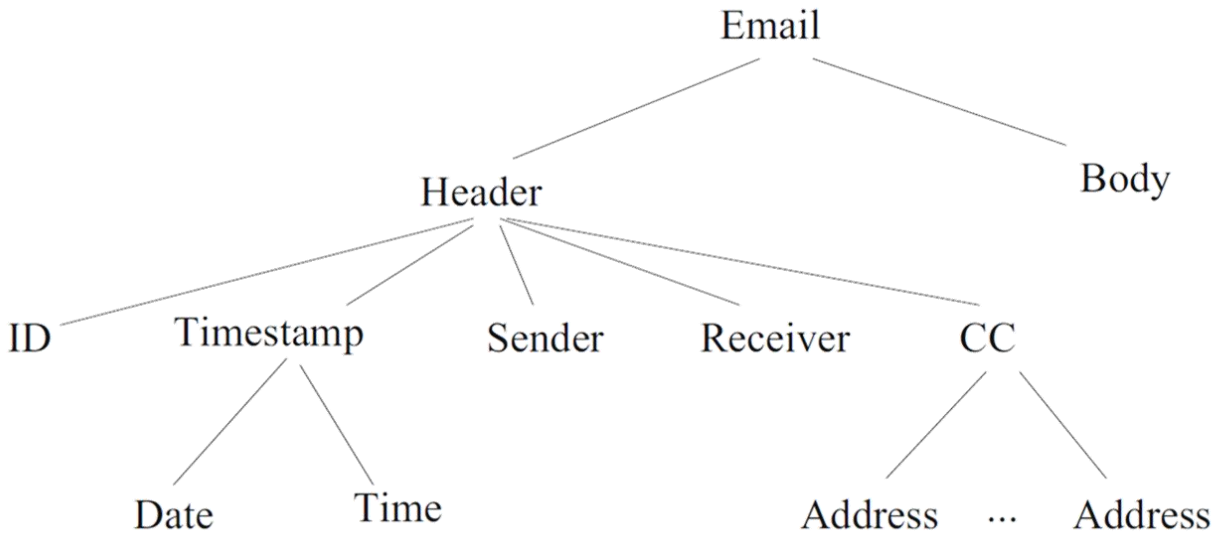
records

custid	sex	is.employed	income	marital.stat	housing.type	num.vehicles	age	state.of.res
2068	F	NA	11300	Married	Homeowner free and clear	2	49	Michigan
2073	F	NA	0	Married	Rented	3	40	Florida
2848	M	True	4500	Never married	Rented	3	22	Georgia
5641	M	True	20000	Never married	Occupied with no rent	0	22	New Mexico
6369	F	True	12000	Never married	Rented	1	31	Florida

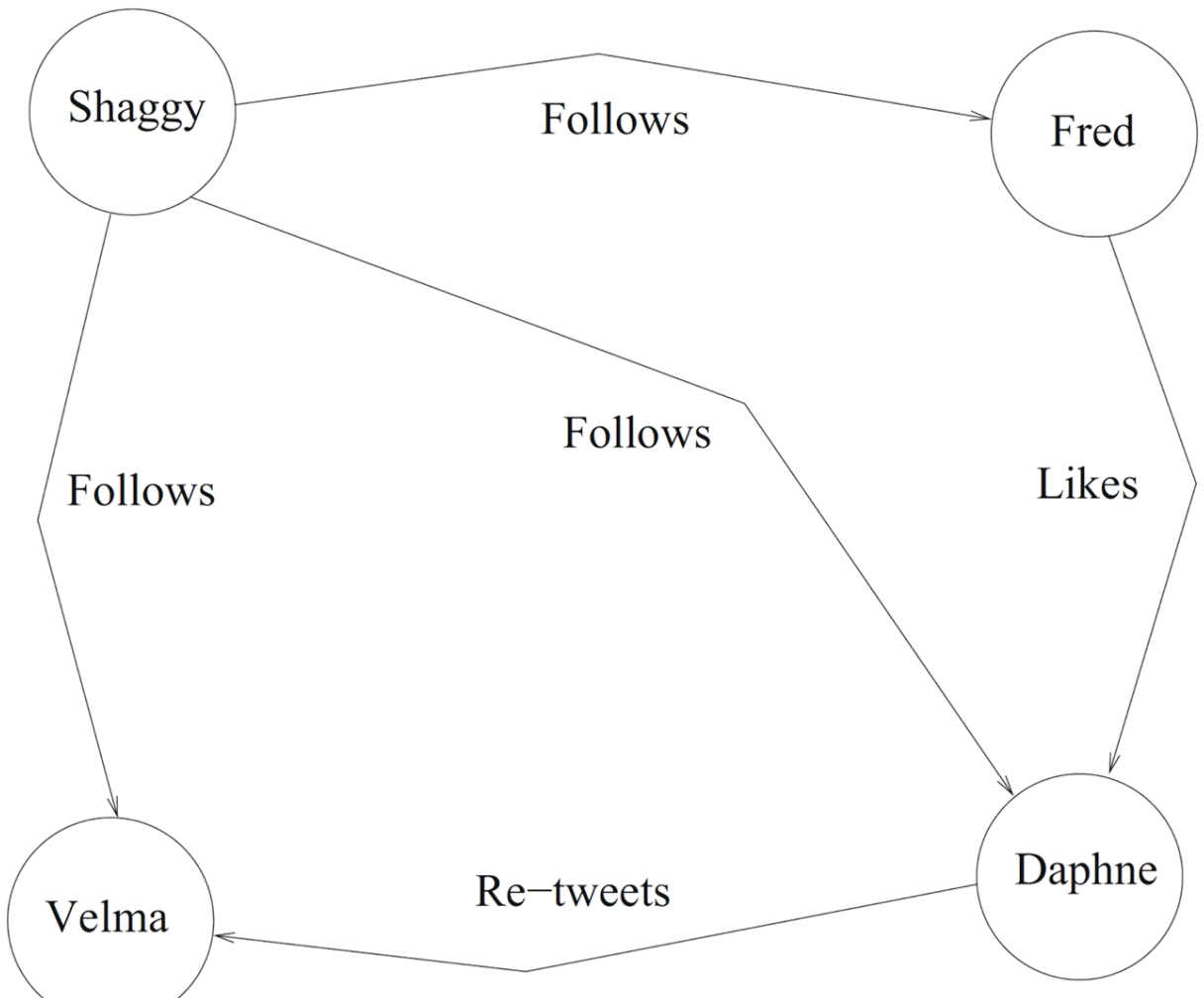
Semi-structured Data

- Data where each record may have a different schema (each record has different sets of attributes)→ some are optional !!!
- Attributes can have one or multiple values
- Example: Emails

```
ID: 19475126.1075855757890
Header:
        timestamp:
                date: Sun, Feb 4 2001
                time: 03:06:00
        sender: robert.benson@enron.com
        receiver: bsunsurf@aol.com
        subject: Rob-are you getting this?
        CC: peter.shipman@axiaenergy.com,
gwadsworth@midf.com
Body: "How about lunch tomorrow?"
```



Social media and graph structures



a node may have an arbitrary number of edges to other nodes !!!→ semistructured

Unstructured Data

- Unstructured data is simply data without labels
  →Organization and compilation of data is data and time consuming
- Structuring the data is key to gaining proper insights !!!!!
- Example? Any sentence we speak ☺
  - It was found that a female with a height between 65 inches and 67 inches had an IQ of 125–130.
  - What if the female is taller or shorter? What is the link with IQ

Metadata

- Basically, data about data
  - Data becomes useful
  - Data can be properly cleaned and preprocessed (how? Soon)
  - Allows sharing and exporting data
  - Allows others to reproduce your results and ensures your credibility

- Should inform about the different attributes in (semi)structured data
- Should provide information about the domain of the data
  - Syntactical metadata: unit, format, range
  - Semantic metadata: what should the data represent?
- Should inform about the source of the data
- Should potentially inform about the quality of the associated data !

Aspect	Structured Data	Semi-Structured Data	Unstructured Data
Definition	Organized data with a fixed schema (rows and columns).	Partially organized with a flexible schema, using tags or markers for structure.	Data with no predefined structure, often free-form.
Storage in Data Warehouse	Stored in relational databases using tables, schemas, and columns. Optimized for SQL-based querying.	Stored in relational or NoSQL databases after transformation (e.g., JSON tables).	Typically not stored directly; if stored, converted into structured or semi-structured forms.
Storage in Data Lake	Stored as structured files (e.g., Parquet, ORC, Avro). Supports large-scale processing.	Stored in native format (e.g., JSON, XML) with minimal preprocessing.	Stored in native format (e.g., images, videos, audio, raw logs).
Examples of Data Formats	CSV, SQL tables (PostgreSQL, MySQL).	JSON, XML, YAML, Avro.	Audio (MP3), video (MP4), images (JPG/PNG), raw logs.
Database Examples	Oracle, MySQL, PostgreSQL, Microsoft SQL Server.	MongoDB, CouchDB, DynamoDB, Snowflake.	Not applicable directly, but data may be referenced by metadata.
Processing Tools	SQL, BI tools like Tableau or Power BI.	SQL with extensions, NoSQL query tools, ETL pipelines.	Big data tools like Hadoop, Spark, and machine learning models.
Use Cases	Transaction processing, business reporting, and analytics.	Log analysis, API data integration, and data interoperability.	Text mining, image recognition, sentiment analysis.
Querying	Schema-dependent querying (e.g., SQL).	Flexible querying with schema-on-read approaches.	Search tools (e.g., ElasticSearch), custom scripts.
Advantages	Easy querying and analytics; high performance for structured data.	Balances flexibility and structure for diverse use cases.	Handles diverse data formats; flexible for future analysis.

Data Sources

- Open
  - NGOs, academia, governmental organizations, known repositories
  - Public, accessible, described (metadata), reusable, complete, timely, properly managed
- Social media (Generation-Z curse☹)
  - Mining Facebook, Twitter, Instagram through proper API
  - Check https://developer.twitter.com/en/docs/twitter-api/v1/data-dictionary/object-model/user
- Multimodal data
  - A.k.a, different forms: images, biomedical data, sensor measurements

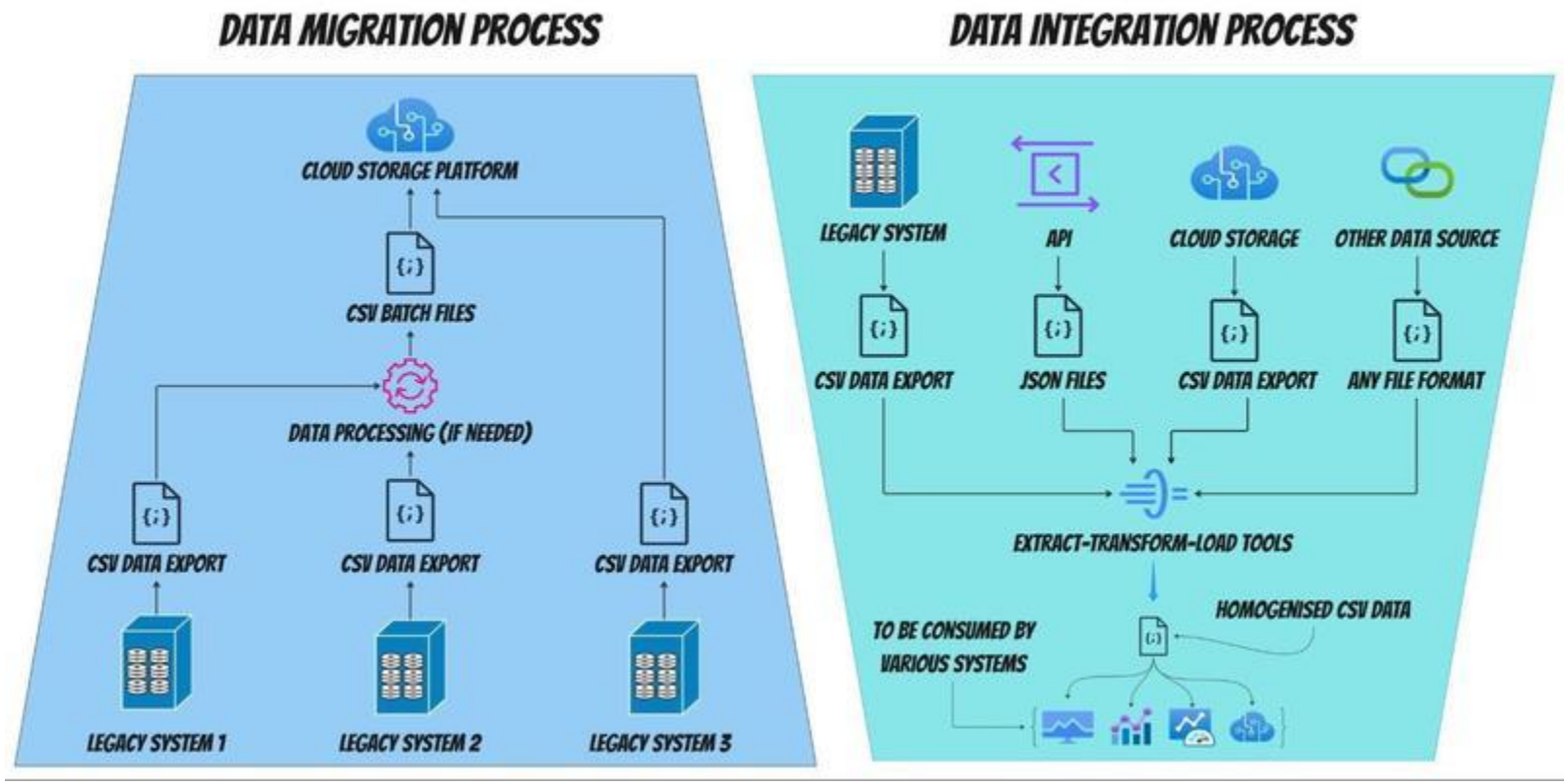
Common file types

CSV

- Comma-Separated Values (CSV) is a simple file format used to store tabular data.
- Example:

  Name, Age, City
  Alice, 30, New York
  Bob, 25, Los Angeles

- Common Uses: Machine learning Data migration, integration, and financial reporting.



Pros and Cons of CSV

PROS

- CSV files are easily made and are compatible with most software and systems.
- They're easy for you and I (humans) to read and edit without specialized software.item
- CSVs allow fast reading and processing due to their plain text and lack of complexity.
- CSV files can represent large datasets in a more compact form than JSON and XML, allowing for smaller files.
- CSV files work well with Microsoft Excel and Google Sheets, which are widely used.

CONS

- They're unsuitable for hierarchical or nested data and lack metadata support.
- CSVs lack data type support, leading to potential misinterpretations.
- You won't be able to include comments or annotations within the data.
- If you use commas within data fields in CSV, it can cause confusion because commas are used for value separation.
- They do NOT work well with APIs because of poor interoperability.

Source: https://sonra.io/csv-vs-json-vs-xml/

What is JSON?

- JavaScript Object Notation (JSON) is a lightweight data-interchange format.
- Perfect for storing data and transmitting it between servers and clients
- Good for configuration files, data storage, and API integrations.
- Supports hierarchical and nested structures (compared to CSV) while being less bulky than XML.
- Example (Tweet):
- Common Uses: Web APIs, configuration files, and NoSQL databases (e.g., MongoDB).

```
"employees":
[
  { "name": "Alice", "age": 30, "city": "New York" },
  { "name": "Bob", "age": 25, "city": "Los Angeles" }
]
```

Array of objects

data structured as a collection of objects


```
{
  "created_at": "Thu Apr 06 15:24:15 +0000 2017",
  "id_str": "850006245121695744",
  "text": "1\/ Today we\u2019re sharing our vision for the future of the Twitter API platform!\nht
  "user": {
    "id": 2244994945,
    "name": "Twitter Dev",
    "screen_name": "TwitterDev",
    "location": "Internet",
    "URL": "https:\/\/dev.twitter.com\/",
    "description": "Your official source for Twitter Platform news, updates & events. Need technic
  }
}
```

Pros and Cons of JSON

PROS

- JSON is widely compatible and adopted across web technologies and APIs.
- Natively supported by JavaScript.
- Compared to XML, JSON has a large community support and a lower learning curve.
- It's more widely adopted in SQL in modern databases than XML and CSV.
- JSON supports hierarchical and nested structures yet is less bulky than XML.

CONS

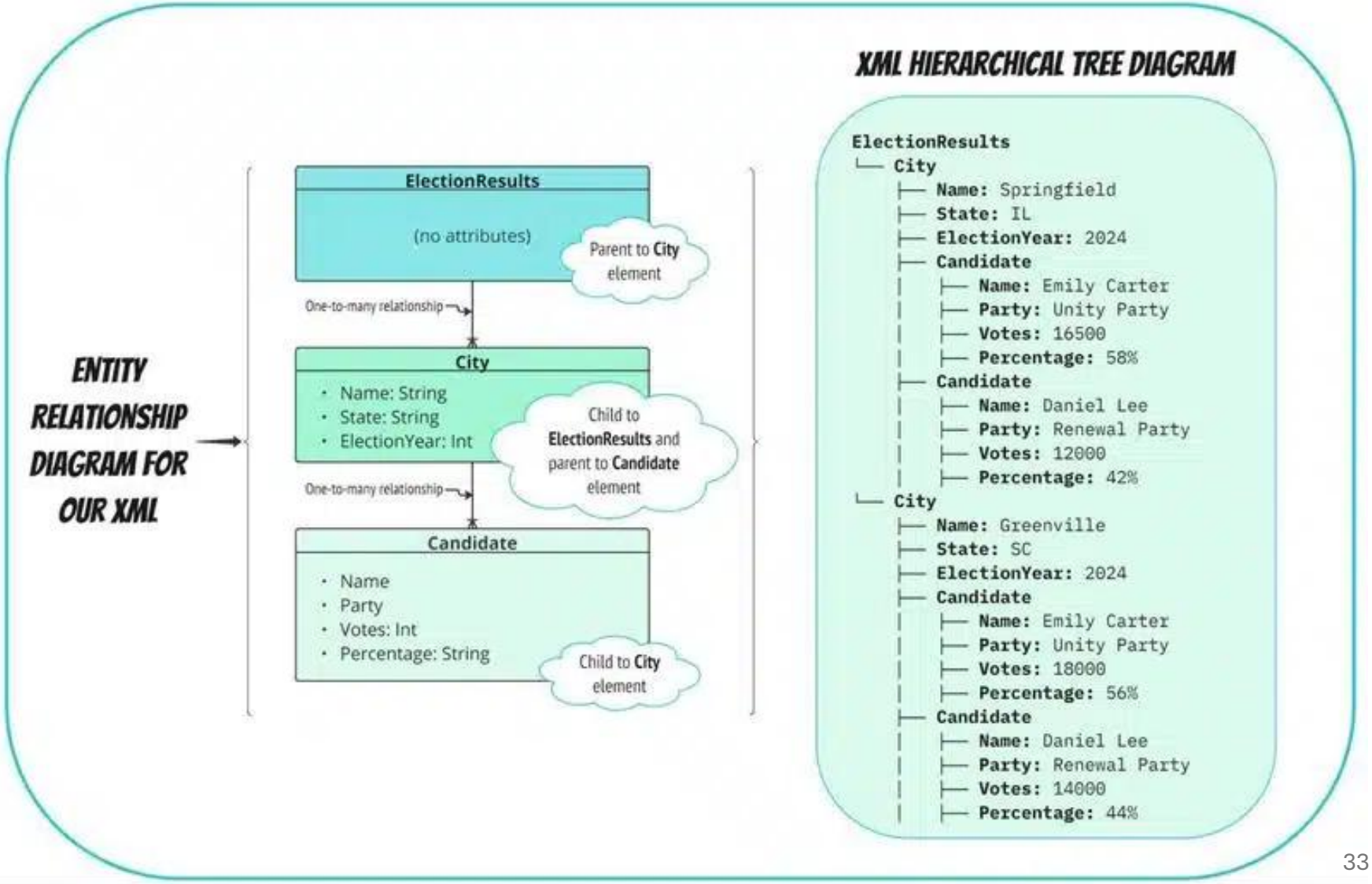
- Less readable (for more complex files) and harder to understand quickly compared to CSV.
- JSON can be slower to parse for large datasets than CSV, but not XML.
- High overhead because of the complexity and nested structure compared to CSV.
- More limited Data Types than XML.
- While JSON schemas exist, they're not widely used and lack the support that XSD enjoys.

What is XML?

- Extensible Markup Language (XML) is a markup language for encoding documents.
- Hierarchical, tag-based approach where each piece of data is enclosed within descriptive tags, forming a tree-like structure that mirrors the relationships between the different elements
- Self-descriptive feature allows for detailed metadata and defined data relationships
- Common Uses: Document storage, web services (SOAP), and extensive metadata.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<ElectionResults>
<City>
<Name>Springfield</Name>
<State>IL</State>
<ElectionYear>2024</ElectionYear>
<Candidate>
<Name>Emily Carter</Name>
<Party>Unity Party</Party>
<Votes>16500</Votes>
<Percentage>58%</Percentage>
</Candidate>
<Candidate>
<Name>Daniel Lee</Name>
<Party>Renewal Party</Party> <Votes>12000</Votes>
<Percentage>42%</Percentage>
</Candidate>
</City>
<City>
<Name>Greenville</Name>
<State>SC</State>
<ElectionYear>2024</ElectionYear>
<Candidate>
<Name>Emily Carter</Name>
<Party>Unity Party</Party>
<Votes>18000</Votes>
<Percentage>56%</Percentage>
</Candidate>
<Candidate>
<Name>Daniel Lee</Name>
<Party>Renewal Party</Party> <Votes>14000</Votes>
<Percentage>44%</Percentage>
</Candidate>
</City>
</ElectionResults>
```

How this example is read?



XML: Pros and Cons

PROS

- XML provides a consistent format for data exchange, great for uniformity across systems with industry standards. Using XSD gives extra validation capabilities as well.
- XML supports hierarchical, data structures like standardized metadata syntax.
- You can take advantage of robust validation with schemas like XSDs and DTDs (or legacy schema).
- XML Defines rich data types, which is good for validation and security with XSD.
- Their structure and capabilities are ideal for SOAP protocol, which is used for secure, complex, and reliable enterprise environments when needed.
- If you pair XML with XSLT templates, you can transform data into other formats, like HTML, other XML schemas, or plain text.

CONS

- Due to extensive tagging, XML files are big and wordy, compared to the compact structure of JSON and the simple, no-tag format of CSV.
- Your storage needs will increase significantly with data size, especially compared to CSV and JSON.
- You'll find XML harder to understand, especially with heavily nested architectures.
- The extensive markup is harder to parse, leading to slower transmission and processing times.
- JSON is more prevalent in RESTful services than XML because of JSON's smaller structure and faster processing.
- While it's still widely used in certain niches, its wordy syntax and larger size make it less efficient for simple data exchange scenarios than JSON.